

1. MASTERBOARD flashen

```
sudo apt install pyenv
pyenv install 3.8.18 (ohne sudo)
```

source bin/activate ist nicht erforderlich, stattdessen wird Python3.8 im make Befehl übergeben (mehr, siehe unten)

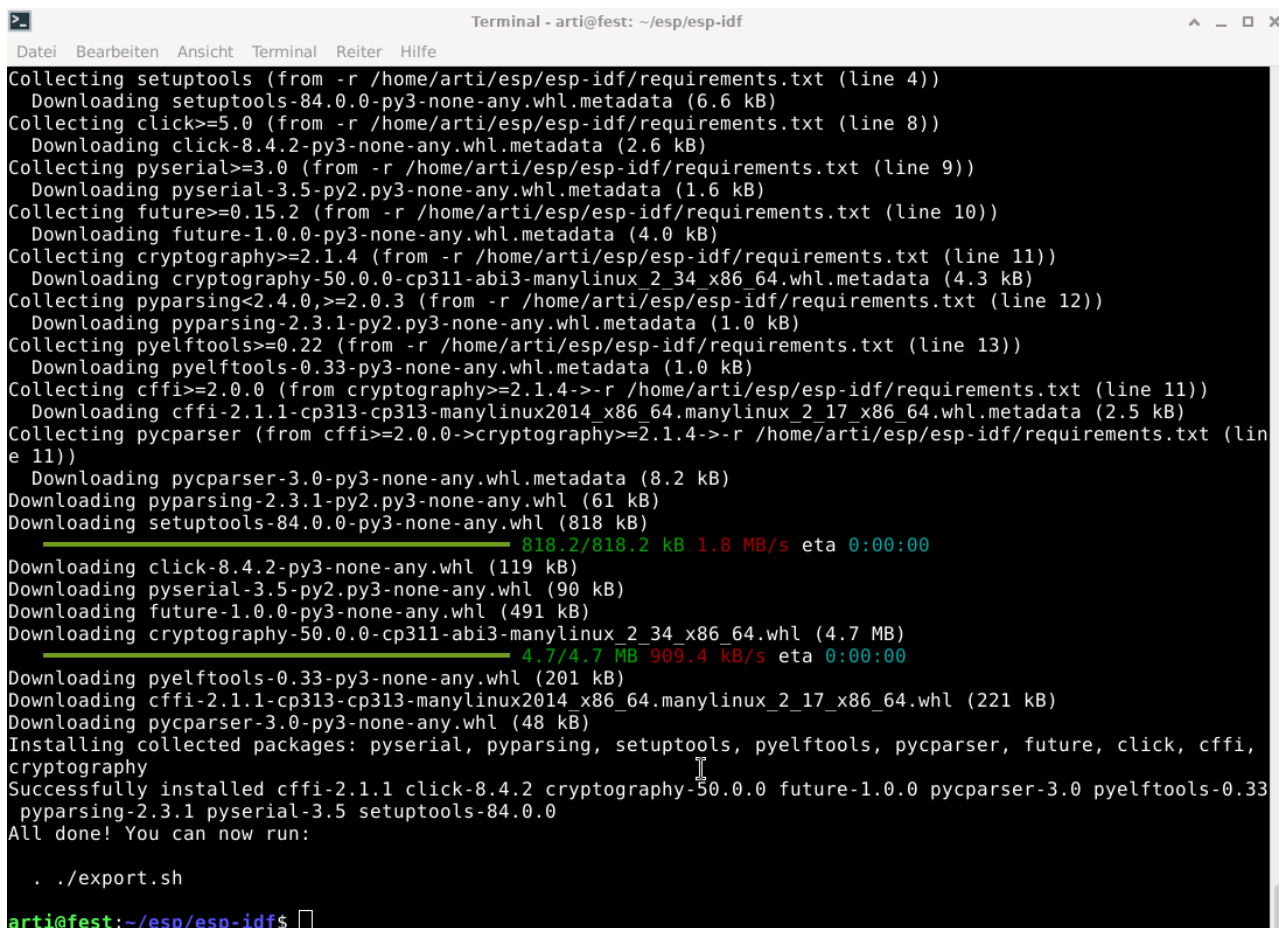
```
sudo apt-get install git wget flex bison gperf python3 python3-pip python3-
setuptools cmake ninja-build ccache libffi-dev libssl-dev dfu-util python3-
vitrualenv
```

```
sudo update-alternatives --install /usr/local/bin/python python /usr/bin/python3
10
```

```
mkdir ~/esp
cd ~/esp
git clone -b v4.0.1 --recursive https://github.com/espressif/esp-idf.git
```

(Achtung, das ist echt viel!)

```
cd ~/esp/esp-idf
./install.sh
```



```
Terminal - arti@fest: ~/esp/esp-idf
Datei Bearbeiten Ansicht Terminal Reiter Hilfe
Collecting setuptools (from -r /home/arti/esp/esp-idf/requirements.txt (line 4))
  Downloading setuptools-84.0.0-py3-none-any.whl.metadata (6.6 kB)
Collecting click>=5.0 (from -r /home/arti/esp/esp-idf/requirements.txt (line 8))
  Downloading click-8.4.2-py3-none-any.whl.metadata (2.6 kB)
Collecting pyserial>=3.0 (from -r /home/arti/esp/esp-idf/requirements.txt (line 9))
  Downloading pyserial-3.5-py2.py3-none-any.whl.metadata (1.6 kB)
Collecting future>=0.15.2 (from -r /home/arti/esp/esp-idf/requirements.txt (line 10))
  Downloading future-1.0.0-py3-none-any.whl.metadata (4.0 kB)
Collecting cryptography>=2.1.4 (from -r /home/arti/esp/esp-idf/requirements.txt (line 11))
  Downloading cryptography-50.0.0-cp311-abi3-manylinux_2_34_x86_64.whl.metadata (4.3 kB)
Collecting pyparsing<2.4.0,>=2.0.3 (from -r /home/arti/esp/esp-idf/requirements.txt (line 12))
  Downloading pyparsing-2.3.1-py2.py3-none-any.whl.metadata (1.0 kB)
Collecting pyelftools>=0.22 (from -r /home/arti/esp/esp-idf/requirements.txt (line 13))
  Downloading pyelftools-0.33-py3-none-any.whl.metadata (1.0 kB)
Collecting cffi>=2.0.0 (from cryptography>=2.1.4->-r /home/arti/esp/esp-idf/requirements.txt (line 11))
  Downloading cffi-2.1.1-cp313-cp313-manylinux2014_x86_64.whl.metadata (2.5 kB)
Collecting pycparser (from cffi>=2.0.0->cryptography>=2.1.4->-r /home/arti/esp/esp-idf/requirements.txt (line 11))
  Downloading pycparser-3.0-py3-none-any.whl.metadata (8.2 kB)
Download pyparsing-2.3.1-py2.py3-none-any.whl (61 kB)
Download setuptools-84.0.0-py3-none-any.whl (818 kB)
818.2/818.2 kB 1.8 MB/s eta 0:00:00
Download click-8.4.2-py3-none-any.whl (119 kB)
Download pyserial-3.5-py2.py3-none-any.whl (90 kB)
Download future-1.0.0-py3-none-any.whl (491 kB)
Download cryptography-50.0.0-cp311-abi3-manylinux_2_34_x86_64.whl (4.7 MB)
4.7/4.7 MB 909.4 kB/s eta 0:00:00
Download pyelftools-0.33-py3-none-any.whl (201 kB)
Download cffi-2.1.1-cp313-cp313-manylinux2014_x86_64.whl (221 kB)
Download pycparser-3.0-py3-none-any.whl (48 kB)
Installing collected packages: pyserial, pyparsing, setuptools, pyelftools, pycparser, future, click, cffi,
cryptography
Successfully installed cffi-2.1.1 click-8.4.2 cryptography-50.0.0 future-1.0.0 pycparser-3.0 pyelftools-0.33
pyparsing-2.3.1 pyserial-3.5 setuptools-84.0.0
All done! You can now run:
. ./export.sh
arti@fest:~/esp/esp-idf$
```

groups (muss dialout drin stehen, falls nicht:)

```
sudo usermod -a -G dialout $USER(ohne sudo, also mit ,su -' statt $USER: ,arti')
```

Neustart oder Abmelden und Anmelden

(sudo dmesg | grep -E "ttyS|ttyUSB|ttyACM") mit angestecktem Board zeigt den Port für espfuse

espfuse.py -p /dev/ttyUSB0 set_flash_voltage 3.3V

```
Terminal - arti@work: ~/Dokumente/Solo12/bin
Datei Bearbeiten Ansicht Terminal Reiter Hilfe
[ 2151.333637] ftdi_sio ttyUSB0: FTDI USB Serial Device converter now disconnected from ttyUSB0
[ 2179.147543] ftdi_sio ttyUSB0: Unable to read latency timer: -32
[ 2179.147867] usb 3-12: FTDI USB Serial Device converter now attached to ttyUSB0
root@work:~# exit
Abgemeldet
(Solo12) arti@work:~/Dokumente/Solo12/bin$ espfuse.py -p /dev/ttyUSB0 set_flash_voltage 3.3V
Warning: DEPRECATED: 'espfuse.py' is deprecated. Please use 'espfuse' instead. The '.py' suffix will be removed in a future major release.
espfuse v5.2.0
Connecting....
Detecting chip type... ESP32
Warning: Deprecated: Command 'set_flash_voltage' is deprecated. Use 'set-flash-voltage' instead.

=== Run "set-flash-voltage" command ===
Enable internal flash voltage regulator (VDD SDIO) to 3.3V.
The following eFuses are burned: XPD_SDIO_FORCE, XPD_SDIO_REG, XPD_SDIO_TIEH.
VDD_SDIO setting complete.

Check all blocks for burn...
idx, BLOCK NAME, Conclusion
[00] BLOCK0 is not empty
(written ): 0x0000000040100000000000004360000a20000bb24dcc36f9680000000000
(to write): 0x000000000000000000000000000000000000000000000000000000000000
(coding scheme = NONE)

This is an irreversible operation!
Type 'BURN' (all capitals) to continue.
BURN
BURN BLOCK0 - OK (all write block bits are set)
Burn postponed eFuses from BLOCK0.
Reading updated eFuses...
Successful.
(Solo12) arti@work:~/Dokumente/Solo12/bin$ su -
Passwort:
root@work:~# dmesg | grep -E "ttyS|ttyUSB|ttyACM"
```

cd master-board/firmware
source setup_esp_idf.bash

cd ~/esp/esp-idf
git checkout 8d1a9c0 # Need to checkout this old version for now
git submodule update --init --recursive

```
Terminal - arti@work: ~/esp/esp-idf
Datei Bearbeiten Ansicht Terminal Reiter Hilfe
M components/expat/expat
M components/json/cJSON
M components/lwip/lwip
M components/mbedtls/mbedtls
M components/mqtt/esp-mqtt
Vorherige Position von HEAD war 4c81978a3e Merge branch 'bugfix/doc_version_links_v4.0' into 'release/v4.0'
HEAD ist jetzt bei 8d1a9c07a0 tools/check python dependencies: make aware of IDF_TOOLS_PATH
(Solo12) arti@work:~/esp/esp-idf$ git submodule update --init --recursive
Submodul 'components/bt/lib' (https://github.com/espressif/esp32-bt-lib.git) für Pfad 'components/bt/lib' in die Konfiguration
eingetragen.
Klone nach '/home/arti/esp/esp-idf/components/bt/lib'...
Submodul-Pfad 'components/bt/lib': '70d6a277d7adb468fc4dbee92a64144569bf080' ausgecheckt
Klone nach '/home/arti/esp/esp-idf/components/coap/libcoap/ext/tinydtls'...
Schwerwiegend: Konnte Basis-URL nicht durch Umleitung aktualisieren:
gefragt nach: https://git.eclipse.org/r/tinydtls/org.eclipse.tinydtls/info/refs?service=git-upload-pack
umgeleitet: https://github.com/eclipse-tinydtls
Schwerwiegend: Klonen von 'https://git.eclipse.org/r/tinydtls/org.eclipse.tinydtls' in Submodul-Pfad '/home/arti/esp/esp-idf/co
mponents/coap/libcoap/ext/tinydtls' fehlgeschlagen.
Fehler beim Klonen von 'ext/tinydtls'. Weiterer Versuch geplant
Klone nach '/home/arti/esp/esp-idf/components/coap/libcoap/ext/tinydtls'...
Schwerwiegend: Konnte Basis-URL nicht durch Umleitung aktualisieren:
gefragt nach: https://git.eclipse.org/r/tinydtls/org.eclipse.tinydtls/info/refs?service=git-upload-pack
umgeleitet: https://github.com/eclipse-tinydtls
Schwerwiegend: Klonen von 'https://git.eclipse.org/r/tinydtls/org.eclipse.tinydtls' in Submodul-Pfad '/home/arti/esp/esp-idf/co
mponents/coap/libcoap/ext/tinydtls' fehlgeschlagen.
Zweiter Versuch 'ext/tinydtls' zu klonen fehlgeschlagen, breche ab.
Schwerwiegend: Fehler bei Rekursion in Submodul-Pfad 'components/coap/libcoap'
Submodul-Pfad 'components/esp_wifi/lib_esp32': 'ebeade52df37b69bd8105542f08fcd1ae4fbb4e8' ausgecheckt
Submodul-Pfad 'components/esptool_py/esptool': 'b4c418a5d90c94863b44c8661b9332cf229b08b7' ausgecheckt
Submodul-Pfad 'components/expat/expat': '968b8cc46dbec47b83318d5f31a8e7907199614b' ausgecheckt
Submodul-Pfad 'components/json/cJSON': '93688cbe72b190300d1be6b98e86b772df9b9ead' ausgecheckt
Submodul-Pfad 'components/lwip/lwip': 'fe9a58cb7abdc74e7a2196b7c4e0a29ff9042f88' ausgecheckt
Submodul-Pfad 'components/mbedtls/mbedtls': '19eb57f7f7b2f4312d497ddbc5f104fc6877c70' ausgecheckt
Submodul-Pfad 'components/mqtt/esp-mqtt': '39118d5182f804edc01ed030c8ba137a9c7f388e' ausgecheckt
(Solo12) arti@work:~/esp/esp-idf$
```

cd master-board/firmware
make all

(wenn es Ärger mit pkg-resources gibt:)

```
sudo apt update
sudo apt install python3.13-venv python3-pip-whl python3-setuptools-whl
```

Global ist setuptools<82.0 schon installiert. Im esp-idf Ordner muss es manuell nachgezogen werden:

```
~/espressif/python_env/idf*_py*_env/bin/python -m pip install
```

```
wget https://dl.espressif.com/dl/xtensa-esp32-elf-linux64-1.22.0-80-g6c4433a-5.2.0.tar.gz
```

```
tar -xzf xtensa-esp32-elf-linux64-1.22.0-80-g6c4433a-5.2.0.tar.gz
```

```
/home/arti/.pyenv/versions/3.8.18/bin/python3 -m pip install
"pyparsing>=2.0.3,<2.4.0" (ohne sudo und für alle fehlenden anderen Pakete!)
```

makefile im firmware Ordner bearbeiten:

```
# Pfad zum alten 5.2.0-Compiler ganz nach vorne hängen, 26_08_01:
export PATH := /home/arti/Dokumente/Solo12/isolierte_Umgebung/firmware/cross-
compiler/xtensa-esp32-elf/bin/:${PATH}
```

```
make all PYTHON=/home/arti/.pyenv/versions/3.8.18/bin/python3
make monitor PYTHON=/home/arti/.pyenv/versions/3.8.18/bin/python3
make clean PYTHON=/home/arti/.pyenv/versions/3.8.18/bin/python3
rm -rf build/
```

make menuconfig funktioniert nicht!

Die Baudrate wird nach der USB Schnittstelle angezeigt:

```
sdkconfig, sdkconfig.defaults bearbeiten (im firmware-Ordner)
main/masterboard_main.c 2000000 in 115200 ändern
```

Flash Befehl:

```
/home/arti/.pyenv/versions/3.8.18/bin/python3
/home/arti/esp/esp-idf/components/esptool_py/esptool/esptool.py --chip esp32 --
port /dev/ttyUSB0 --baud 460800 write_flash -z --flash_mode dio --flash_freq 40m
--flash_size detect 0x1000 build/bootloader/bootloader.bin 0x8000
build/partition_table/partition-table.bin 0x10000 build/masterboard.bin
```

flash.sh ist ein Skript mit dem obigen Befehl

```
Terminal - arti@fest: ~/Dokumente/Solo12/isolierte_Umgebung/firmware
Datei Bearbeiten Ansicht Terminal Reiter Hilfe
File "/home/arti/esp/esp-idf/tools/idf_monitor.py", line 144, in stop
    self._cancel()
File "/home/arti/esp/esp-idf/tools/idf_monitor.py", line 204, in _cancel
    fcntl.ioctl(self.console.fd, termios.TIOCSSTI, b'\0')
OSError: [Errno 5] Input/output error
make: *** [/home/arti/esp/esp-idf/components/esptool_py/Makefile.projbuild:110: monitor] Fehler 1
arti@fest:~/Dokumente/Solo12/isolierte_Umgebung/firmware$ make monitor PYTHON=/home/arti/.pyenv/versions/3.8.18/bin/python3
Toolchain path: /home/arti/Dokumente/Solo12/isolierte_Umgebung/firmware/cross-compiler/xtensa-esp32-elf/bin//xtensa-esp32-elf-gcc
Toolchain version: crosstool-ng-1.22.0-80-g6c4433a
Compiler version: 5.2.0
Python requirements from /home/arti/esp/esp-idf/requirements.txt are satisfied.
MONITOR
--- idf_monitor on /dev/ttyUSB0 115200 ---
--- Quit: Ctrl+] | Menu: Ctrl+T | Help: Ctrl+T followed by Ctrl+H ---
etg (0-
00$0F&0SHPH0001BvU000URUI00E:01d )0Ie1==0U0
@me0,(00*020HB0+0$'0300b0)KjEqi000XdEq000i0_00K000',A00E0'00`p_!:00`
0+:030000,00)K02
0
000 L&000+0C!0+0$'0400 b0G0&0TC!0
00*
000U 0HB0+0$'040b:h7Mj
A00 000j
I (31) boot: ESP-IDF v4.0-dev-615-g8d1a9c07a0-dirty 2nd stage bootloader
I (32) boot: compile time 15:03:24
I (33) boot: Enabling RNG early entropy source...
I (38) boot: SPI Speed      : 40MHz
I (42) boot: SPI Mode       : DIO
I (46) boot: SPI Flash Size : 4MB
I (50) boot: Partition Table:
I (54) boot: ## Label                Usage            Type ST Offset   Length
I (61) boot: 0 nvs                   WiFi data        01 02 00009000 00006000
I (69) boot: 1 phy_init                RF data          01 01 0000f000 00001000
I (76) boot: 2 factory                factory app       00 00 00010000 00100000
I (84) boot: End of partition table
I (88) esp_image: segment 0: paddr=0x00010020 vaddr=0x3f400020 size=0x1e4bc (124092) map
I (140) esp_image: segment 1: paddr=0x0002e4e4 vaddr=0x3ffb0000 size=0x01b2c ( 6956) load
I (143) esp_image: segment 2: paddr=0x00030018 vaddr=0x400d0018 size=0x71198 (463256) map
0x400d0018: _stext at ???:
I (309) esp_image: segment 3: paddr=0x000a11b8 vaddr=0x3ffb1b2c size=0x01520 ( 5408) load
I (311) esp_image: segment 4: paddr=0x000a26e0 vaddr=0x40080000 size=0x00400 ( 1024) load
0x40080000: _WindowOverflow4 at /home/arti/esp/esp-idf/components/freertos/xtensa_vectors.S:1779
I (316) esp_image: segment 5: paddr=0x000a2ae8 vaddr=0x40080400 size=0x120bc ( 73916) load
I (366) boot: Loaded app from partition at offset 0x10000
I (366) boot: Disabling RNG early entropy source...
I (367) cpu_start: Pro cpu up.
I (370) cpu_start: Application information:
I (375) cpu_start: Project name:      masterboard
I (380) cpu_start: App version:      1
I (385) cpu_start: Compile time:     Aug 3 2026 15:03:26
I (391) cpu_start: ELF file SHA256:  b4ac7f822949bafd...
I (397) cpu_start: ESP-IDF:         v4.0-dev-615-g8d1a9c07a0-dirty
I (404) cpu_start: Starting app cpu, entry point is 0x40081220
0x40081220: call_start_cpu1 at /home/arti/esp/esp-idf/components/esp32/cpu_start.c:267
I (0) cpu_start: App cpu up.
I (414) heap_init: Initializing. RAM available for dynamic allocation:
I (421) heap_init: At 3FFAE6E0 len 00001920 (6 KiB): DRAM
I (427) heap_init: At 3FFB9918 len 000266E8 (153 KiB): DRAM
I (433) heap_init: At 3FFE0440 len 00003AE0 (14 KiB): D/IRAM
I (440) heap_init: At 3FFE4350 len 00018CB0 (111 KiB): D/IRAM
```

Oben ist der Ausdruck des make monitor bei richtiger Baudrateneinstellung.

Es kommen über make monitor KEINE Echtzeitdaten vom Masterboard, nur kritische Meldungen!

2. MASTERBOARD TESTEN (Ethernet, WLAN: siehe unten TODO)

```
sudo ip a (Interfacename finden)
sudo ip link set dev „INTERFACE“ arp off

sudo vi /etc/sysctl.d/99-disable-ipv6.conf
99-disable-ipv6.conf:
    net.ipv6.conf."INTERFACE".disable_ipv6 = 1
sudo sysctl --system
```

Screenshot:

```
root@work:~# vi /etc/sysctl.d/99-disable_ipv6_enp8s0.conf
root@work:~# sysctl --system
* /usr/lib/sysctl.d/10-coredump-debian.conf wird angewendet ...
* /usr/lib/sysctl.d/50-bubblewrap.conf wird angewendet ...
* /usr/lib/sysctl.d/50-default.conf wird angewendet ...
* /usr/lib/sysctl.d/50-pid-max.conf wird angewendet ...
* /etc/sysctl.d/99-disable_ipv6_enp8s0.conf wird angewendet ...
kernel.core_pattern = core
kernel.unprivileged_usersns_clone = 1
kernel.sysrq = 0x01b6
kernel.core_uses_pid = 1
net.ipv4.conf.default.rp_filter = 2
net.ipv4.conf.eno1.rp_filter = 2
net.ipv4.conf.enp4s1.rp_filter = 2
net.ipv4.conf.enp8s0.rp_filter = 2
net.ipv4.conf.lo.rp_filter = 2
net.ipv4.conf.wlp7s0.rp_filter = 2
net.ipv4.conf.default.accept_source_route = 0
net.ipv4.conf.eno1.accept_source_route = 0
net.ipv4.conf.enp4s1.accept_source_route = 0
net.ipv4.conf.enp8s0.accept_source_route = 0
net.ipv4.conf.lo.accept_source_route = 0
net.ipv4.conf.wlp7s0.accept_source_route = 0
net.ipv4.conf.default.promote_secondaries = 1
net.ipv4.conf.eno1.promote_secondaries = 1
net.ipv4.conf.enp4s1.promote_secondaries = 1
net.ipv4.conf.enp8s0.promote_secondaries = 1
net.ipv4.conf.lo.promote_secondaries = 1
net.ipv4.conf.wlp7s0.promote_secondaries = 1
net.ipv4.ping_group_range = 0 2147483647
net.core.default_qdisc = fq_codel
fs.protected_hardlinks = 1
fs.protected_symlinks = 1
fs.protected_regular = 2
fs.protected_fifos = 1
vm.max_map_count = 1048576
kernel.pid_max = 4194304
net.ipv6.conf.enp8s0.disable_ipv6 = 1
```

libmaster_board_sdk_pywrap erstellen um example.py, listener.py oder com_analyser.py auszuführen:

```
apt install libboost-all-dev libboost-python-dev
apt install libcatch2-dev
```

```
im master_board_sdk/ -Ordner:
cmake -DBUILD_PYTHON_INTERFACE=ON -DCMAKE_BUILD_TYPE=Release
-DPYTHON_EXECUTABLE=$(which python3) ..
```

```
sollte libboost (cmake/boost.cmake) meckern:
set(BOOST_PYTHON_NAME "python313") einfügen vor
find_package(Boost ${BOOST_PYTHON_REQUIRED} COMPONENTS ${BOOST_PYTHON_NAME})
```

make
(dann sollte im build Ordner die Datei libmaster_board_sdk.so entstanden sein)

in com_analyzer.py und example.py folgende Zeile einfügen, wenn libmaster...so vermisst wird:

```
sys.path.insert(0,  
"/home/arti/Dokumente/Solo12/isolierte_Umgebung/master_board_sdk/build/")  
und dann die libmaster...so importieren (Achtung: Namen müssen stimmen! OHNE .so)
```

```
im Skript:  
BUILD_PATH = "/home/arti/master-board/sdk/master_board_sdk/build"  
  
if os.path.exists(BUILD_PATH):  
    sys.path.append(BUILD_PATH)  
else:  
    print(f"WARNUNG: Pfad {BUILD_PATH} existiert nicht!")
```

```
sudo python3.13 example.py -i enp8s0
```

IP-Adresse des Masterboards:

```
components/direct_ethernet/direct_ethernet.h
```

alle letzten Ergebnisse basieren auf dem Ordner: isolierte_Umgebung

2.a. MOTORWARE installieren

motorware bei TI runterladen rob.schierer@yahoo.de PW: St*****!

Ansonsten die Anleitung von hier nehmen:

```
https://open-dynamic-robot-initiative.github.io/udriver_firmware/  
build_instructions.html#install-code-composer-studio
```

(

How to Build and Flash the Firmware

Get the Code

Simply clone the firmware repository (including submodules):

```
$ git clone --recursive  
https://github.com/open-dynamic-robot-initiative/udriver_firmware.git
```

The default configuration is assuming the *Antigravity 4004 300kv* motors. In case you are using different motors, change the config_f28069m_drv8305 submodule to the appropriate branch.

Get MotorWare

Our software is based on Texas Instruments' [MotorWare](#) library which can be downloaded from their website. We made some modifications to the original MotorWare which are needed to build our software. We provide patch files

motorware_1_01_00_18_patch with these changes, they are included in the directories firmware_can/firmware_spi (note that they differ for the two firmware versions).

Use the following steps to set up MotorWare for the project:

1. Download MotorWare v1.01.00.18 from the [MotorWare](#) website.
2. Install it to extract the source. For Linux users: Unfortunately TI only provides a Windows installer. There is not really an installation required, you only need the source files. You can get them by either installing on a Windows machine or using an emulator like Wine on Linux and then copy the files from the installation directory. Search for the directory containing the sw directory and copy it into the directory firmware_X/motorware (where X is either "can" or "spi", depending on which version you want to use). The structure should be like this:

```
udriver_firmware/firmware
├── amd_motorware_ext
├── config_f28069m_drv8305
├── firmware_X
│   ├── motorware                <-- Add motorware here
│   │   ├── docs
│   │   ├── eclipse
│   │   ├── mw_explorer
│   │   └── sw
│   └── mw_dual_motor_torque_ctrl
```

3. Finally go into the motorware directory and apply the patch file:

```
$ patch -p1 < ../motorware_1_01_00_18.patch
```

)

Es hat alles auf Anhieb funktioniert.

Im Ordner: udriver ist alles gepatch und fertig

2.b. Udriver programmieren

CCS_21.0.0.00014_linux bei TI runterladen (Anmeldung erforderlich)

entpacken...

ccs_setup_21.0.0.00014.run als User ausführen

apt install libusb-0.1-4 war nötig

libpython3.9.so.1.0 Fehlermeldung bleibt während der Installation oder:

```
ln -s /usr/lib/x86_64-linux-gnu/libpython3.13.so.1.0 /usr/lib/x86_64-linux-gnu/libpython3.9.so.1.0
```

leitet die 3.9 er Version auf die 13er um

nur C2000 32-bit Real-time MCUs auswählen

Hier die read.me von CCS:

Code Composer Studio for Linux README

If you are using an Offline CCS Installer, the zip file in which it is distributed must first be extracted:

1. unzip CCSxx.x.x.xxxxx_linux.zip
2. cd CCS_xx.x.x.xxxxx_linux

Once extraction has successfully completed, proceed with the installation steps below.

To install Code Composer Studio:

1. Execute ccs_setup_xx.x.x.xxxxx.run
2. Choose where you want to install.
3. Select the components you wish to install.

In order to complete the installation, please run the script `${ccsinstalldir}/install_scripts/install_drivers.sh` as administrator.

CCS starten mit:

als user: `~/ti/ccs2100/ccs/theia/./csstudio`

~~3. UM7 Anschluss und IBN~~

~~Arduino IDE installieren~~

~~sudo apt install arduino~~

~~Werkzeuge > Board > Boardverwalter (oder das Icon links). Suche nach **esp32** und installiere das Paket „esp32“ von Espressif Systems~~

~~Das ist wirklich viel (~3GB)!~~

~~Werkzeuge > Board dein spezifisches Modul aus (meistens **ESP32 Dev Module**)~~

~~Werkzeuge > Port~~

~~4. Arduino IDE mit GEANY einrichten~~

~~Strg + , in der Arduino IDE~~

~~Externen Editor verwenden~~

~~Geany erkennt .ino-Dateien standardmäßig oft nicht sofort als C/C++ Code. So korrigieren Sie das:~~

~~Klicken Sie im oberen Menü auf **Dokument** → **Dateityp festlegen** → **Programmiersprachen** → **C++-Quelltext**.~~

~~Autovervollständigen aktivieren:~~

~~Gehen Sie auf **Bearbeiten** → **Einstellungen**.~~

- ~~• Wählen Sie links **Editor** und dann den Reiter **Fertigstellung** (oder **Vervollständigung**).~~
- ~~• Setzen Sie unbedingt den Haken bei "**Alle Wörter im Dokument vervollständigen**". [\[1\]](#)~~

5. Masterboard programmieren:

im firmware-Ordner :

```
. setup_esp_idf.bash (entspricht dem: source setup_esp_idf.bash)
```

```
geany &
```

```
./flash.sh (Achtung im Skript wird nach dem Flashvorgang der Ordner build/uart_imu gelöscht, damit die uart_imu.c geupdatet wird beim make)
```

```
sudo cd /home/arti/Dokumente/Solo12/isolierte_Umgebung/master_board_sdk/example
```

```
sudo python3 example.py -i enp8s0
```

```
sudo minicom -b 115200 -D /dev/ttyUSB0
```

Misc:

NVIDIA Treiber:

```
apt install build-essential gcc dirmngr ca-certificates software-properties-common apt-transport-https dkms curl gpg -y
```

```
apt install -y linux-headers-$(uname -r)
```

```
apt install nvidia-driver
```